Desenvolvimento de Sistemas para Dispositivos Móveis

Formulário, tema, acessibilidade, navegação, layout adaptativo e testes

DIM0524 — Turma 01 · Sprint 1 · parte 2 (vídeo)

Prof. Fernando · UFRN · 2026.2

Errata do material já publicado

Onde	O que corrigir
Slides 03, "Android Studio" e "Como rodar hoje"; leitura pte1, cap. 2	O Android Studio do laboratório (Ladybug, 2024) **não abre** o exemplo: o AGP 9.1 exige o Panda 2 ou mais novo. Até a atualização, use o **Codespaces**: Android Studio com `@Preview` e app com Hot Reload, no navegador (próximo slide)
Slides 03, "`@Preview` — ver sem rodar"	O import certo é `androidx.compose.ui.tooling.preview.Preview`; o antigo, `org.jetbrains.compose...`, não compila com o exemplo
Slides 03, "Passo 5"	`MaterialTheme(colorScheme = ...)` troca as cores dos componentes, mas **não pinta o fundo**: falta uma `Surface` na raiz. E o `App()` do exemplo ficava sempre claro: corrigido (Passo 5, a seguir)
Cronograma	Entrega da Sprint 1 adiada para **16/10 (sexta), 23:59**. A aula de 23/09 foi cancelada; em 28 e 30/09, no lugar das apresentações, houve uma *daily meeting* online com cada grupo. O semestre passa a ter só mais uma sprint, a de novembro

Ambiente: o laboratório e o Codespaces

O exemplo usa AGP 9.1, que exige o Android Studio Panda 2 (2025.3.2) ou mais novo. No laboratório, e em computadores em que o Android Studio ou o emulador ficam pesados, use o **Codespaces** do repositório: o trabalho roda na nuvem, e a máquina só exibe o navegador.

Precisa de	Onde
Construir telas e rodar os testes	alvo desktop com Hot Reload, só com o JDK: `mise run app`
`@Preview`	Android Studio **dentro do Codespace**: `mise run studio`
App em janela de celular	no Codespace: `mise run app:celular`
Ver no Android	APK gerado no Codespace (`mise run apk`), instalado no emulador do laboratório ou num **celular**

No Codespace, tudo aparece na **porta 6080** (área de trabalho no navegador, senha `vscode`). Pare o Codespace ao terminar: o uso gratuito mensal é limitado.

> Conferido num Codespace novo em 29/09. Na primeira vez, o Android Studio pergunta sobre estatísticas e se você confia no projeto (**Trust Project**); o primeiro `@Preview` pede **Build & Refresh** (alguns minutos). O VS Code não tem o renderizador do `@Preview`.

O APK no emulador do laboratório

```
./gradlew :composeApp:assembleDebug
# composeApp/build/outputs/apk/debug/composeApp-debug.apk
```

- Baixe pelo explorador do VS Code e **arraste para o emulador**
- Funciona em qualquer versão do Android Studio (imagem Android 7 ou mais nova)
- A **chave de debug está no repositório** (`keystore/`): um APK de qualquer máquina atualiza o de outra

Já tinha o app de outra máquina? `INSTALL_FAILED_UPDATE_INCOMPATIBLE` → desinstale uma vez:

```
adb uninstall br.ufrn.exemplo.tarefas
```

Onde paramos

No vídeo da parte 1, **passos 1 a 3**, tudo em `App.kt`:

```
@Composable com estado: remember + mutableStateOf
CartaoTarefa: recebe a tarefa, devolve o evento (estado elevado)
LazyColumn com key · val + copy + reatribuição
```

Hoje	O que entra
4 e 5	Formulário com validação · tema claro e escuro
6	Acessibilidade — e o código se divide em três arquivos
7	Navegação: rotas tipadas, argumento, deep link
8	Janela compacta × larga
9 e 10	Testes de regra e de interface · o ambiente em tasks do `mise`

Passo 4

Formulário com validação

Passo 4 — formulário

```
var texto by remember { mutableStateOf("") }
var proximoId by remember { mutableStateOf(3) }

val valido = texto.isNotBlank()          // derivado do estado, não guardado

Row(verticalAlignment = Alignment.CenterVertically) {
    OutlinedTextField(
        value = texto,
        onValueChange = { texto = it },
        label = { Text("Nova tarefa") },
        isError = texto.isNotEmpty() && !valido,
        modifier = Modifier.weight(1f),
    )
    Button(onClick = { tarefas = tarefas + Tarefa(proximoId++, texto.trim()); texto = "" },
           enabled = valido) { Text("Adicionar") }
}
```

- O botão nasce desabilitado; espaços não habilitam; um título habilita
- `valido` é recalculado a cada recomposição: não existe um `botaoAtivo` para esquecer de atualizar

📖 **Ref.** Compose — campos de texto

Passo 5

Material 3: tema claro e escuro

Passo 5 — o tema segue o sistema

```
@Composable
fun App() {
    // `MaterialTheme { }` sem `colorScheme` é sempre claro.
    val cores = if (isSystemInDarkTheme()) darkColorScheme() else lightColorScheme()

    MaterialTheme(colorScheme = cores) {
        Surface(Modifier.fillMaxSize()) {                // pinta o fundo e a cor do texto
            Column(Modifier.safeDrawingPadding().padding(16.dp)) { /* ... */ }
        }
    }
}
```

Sem	O que acontece
`colorScheme`	o app fica claro, mesmo com o sistema escuro
`Surface`	os componentes escurecem, o **fundo e o texto solto não**
`safeDrawingPadding()`	o título fica embaixo da barra de status (Android desenha de ponta a ponta)

No emulador: `adb shell cmd uimode night yes`. O `MainActivity` chama `enableEdgeToEdge()`, que ajusta a cor dos ícones da barra de status.

Passo 6

Acessibilidade: a linha inteira é a caixa de seleção

Passo 6 — o código se divide

Com duas telas, o código passa a ser dividido em três arquivos:

Arquivo	O que tem
`Tarefa.kt`	modelo e **regras puras** (`tituloValido`, `comNova`, `alternando`)
`Telas.kt`	`TelaLista`, `CartaoTarefa`, `FormularioTarefa`, previews
`App.kt`	tema e estado (`Conteudo`); no Passo 7, as rotas

> Regras fora do Compose são funções comuns: testáveis com `kotlin.test`, sem montar tela (Passo 9).

Antes e depois

Antes 2 nós

```
Node #3  Role = 'Checkbox'
         ToggleableState = 'Off'
Node #4  Text = '[Estudar Compose]'
```

Caixa sem nome, texto sem ação

Passo 6 1 nó

```
Node #9  Role = 'Checkbox'
         Text = '[Estudar Compose]'
         ToggleableState = 'Off'
         MergeDescendants = 'true'
```

A linha inteira: nome, papel, estado

```
Row(Modifier.toggleable(value = tarefa.feita, role = Role.Checkbox,
                        onValueChange = { onAlternar() })) {
    Checkbox(checked = tarefa.feita, onCheckedChange = null) // o clique é da linha
    Text(tarefa.titulo)
}
```

O que a rubrica pede

Critério	Como
Descrições de conteúdo	`contentDescription` no que não tem texto; `null` no decorativo
Contraste verificado	cores pelos papéis do tema; Accessibility Scanner
Alvos ≥ 48 dp	componentes Material 3 já garantem; cuidado com `clickable` à mão
Leitor de tela testado	TalkBack: deslizar para navegar, toque duplo para ativar

Mais: títulos com `semantics { heading() }`, texto em `sp`.

> Descrição diz **o que faz** ("Apagar tarefa"), não **como é** ("ícone de lixeira").

Passo 7

Navegação: um grafo, com pilha de retorno

Rotas tipadas

```
@Serializable object Lista
@Serializable data class Detalhe(val id: Int)

NavHost(navController, startDestination = Lista) {
    composable<Lista> {
        TelaLista(/* ... */ onAbrir = { id -> navController.navigate(Detalhe(id)) })
    }
    composable<Detalhe> { entrada ->
        val rota = entrada.toRoute<Detalhe>()
        TelaDetalhe(tarefas.find { it.id == rota.id }, /* ... */
                    onVoltar = { navController.popBackStack() })
    }
}
```

- Argumento é **propriedade**, conferida pelo compilador
- Telas recebem **lambdas**, não o `NavController`
- Passe o **id**, não o objeto; o estado mora **acima** do `NavHost`

Pilha de retorno e deep link

- `navigate` empilha; `popBackStack` desempilha; o **Voltar do sistema** também (conferido)
- No desktop não há Voltar do sistema: botão na tela

```
composable<Detalhe>(
    deepLinks = listOf(navDeepLink<Detalhe>(basePath = "tarefas://tarefa")),
)
```

```
<intent-filter>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.DEFAULT" />
    <category android:name="android.intent.category.BROWSABLE" />
    <data android:scheme="tarefas" android:host="tarefa" />
</intent-filter>
```

```
adb shell am start -a android.intent.action.VIEW -d "tarefas://tarefa/2"
```

> Aberto por deep link, o Voltar leva para a lista, não para fora do app.

Passo 8

Responsividade: a largura da janela decide

Classes de tamanho de janela

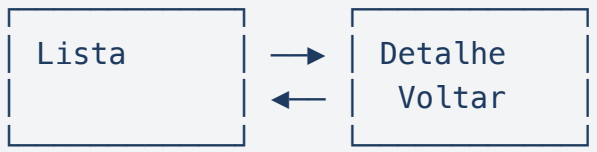
Classe	Largura	Exemplos
Compacta	< 600 dp	celular em pé
Média	600 a 839 dp	celular deitado, tablet pequeno
Expandida	≥ 840 dp	tablet deitado, desktop

```
val largo = currentWindowAdaptiveInfo().windowSizeClass
    .isWidthAtLeastBreakpoint(WindowSizeClass.WIDTH_DP_MEDIUM_LOWER_BOUND)
```

- **Janela**, não aparelho: tela dividida, dobrável, janela desktop
- **Largura**, não orientação: tablet em pé é mais largo que celular deitado
- Conferido: 599 px → uma tela; 600 → dois painéis

Lista e detalhe

Compacta `navega`



Larga `seleciona`



- As **mesmas telas** nos dois modos; muda quem as organiza
- `App()` decide, `Conteudo(largo)` monta: o teste escolhe o modo
- `Surface` e `safeDrawingPadding()` já estão na raiz desde o Passo 5

Passo 9

Testes: `kotlin.test` e teste de interface

Duas camadas

Regras puras `kotlin.test`

```
@Test
fun tituloEmBrancoNaoEValido() {
    assertFalse(tituloValido("   "))
}
```

3 testes em 0,018 s

Interface `runComposeUiTest`

```
@Test
fun linhaInteiraEUmaCaixaDeSelecao() =
  runComposeUiTest {
    setContent { Conteudo(largo = false) }
    onNodeWithText("Estudar Compose")
      .assertIsToggleable().assertIsOff()
    onNodeWithText("Estudar Compose")
      .performClick()
    onNodeWithText("Estudar Compose")
      .assertIsOn()
  }
```

`./gradlew :composeApp:desktopTest` — sem emulador. O teste usa a **mesma árvore** do leitor de tela.

No CI

O workflow da Sprint 0 (build, `ktlint`, `detekt`) ganha os testes:

```
- run: sudo apt-get update && sudo apt-get install -y libgl1 libegl1 libfontconfig1
- run: ./gradlew :composeApp:desktopTest
```

- Os testes de interface **não abrem janela**, mas o Skia precisa de bibliotecas gráficas
- Sem elas (conferido num container Linux mínimo): `UnsatisfiedLinkError: ... libGL.so.1`
- Com elas: os 8 testes passam, sem tela

> Dois nós com o mesmo texto? `onNodeWithText` falha com *Expected exactly '1' node but found '2'*. Use `onAllNodesWithText(...)[0]`.

Oficina: testes para o projeto

1. Regras em **funções puras**, cobertas com `kotlin.test`
2. **Uma tela, um teste**: monta com dados fixos, confere o que aparece
3. **Formulário**: um caso inválido, um válido
4. **Navegação**: abre, age, volta, confere
5. **Acessibilidade**: `assertIsToggleable`, `onNodeWithContentDescription`

> Tela testável = tela que recebe estado e devolve eventos.

Passo 10 — o ambiente em tasks

```
# mise.toml, na raiz do repositório
[tools]
java = "temurin-21"

[tasks.app]
dir = "exemplos/tarefas-compose"
run = '''
if [ -n "$CODESPACES" ]; then DISPLAY=:1 ./gradlew :composeApp:hotRunDesktop --auto
else ./gradlew :composeApp:hotRunDesktop --auto; fi
'''
```

```
mise tasks            # app, app:celular, test, apk, apk:instalar, studio
mise run test         # == ./gradlew :composeApp:desktopTest
mise run studio       # Android Studio no Codespace, para o @Preview
```

O mesmo comando dos passos anteriores, com nome — e o detalhe do ambiente resolvido: no Codespace, a janela vai para a área de trabalho do navegador.

📖 **Ref.** mise — tasks

No MUSI

O projeto de referência ligou o alvo Android nesta sprint (`app/`):

O que a rubrica pede	Onde ver
Rotas tipadas com argumento	`Acervo` e `DetalheDaObra(val id: String)` — id de texto, do domínio
Deep link	`musi://obra/{id}`, no grafo e no manifesto
Duas larguras	estreita navega; larga mostra acervo e obra lado a lado
Tema	`TemaMusi`, com `isSystemInDarkTheme()`
Formulário validado	a tela usa as regras do módulo `shared`, as mesmas da API
Testes de interface	8, no alvo desktop, rodando no CI

`github.com/fmarquesfilho/musi`

> `./gradlew build` roda também `testDebugUnitTest`, que executa os testes de `commonTest` numa JVM sem Android: teste de interface quebra ali. O lugar deles é o alvo desktop.

Entrega da Sprint 1 — 16/10, 23:59

Critério	Peso
Telas do MVP, com componentes próprios	25%
Navigation Compose: rotas tipadas, argumentos, ≥ 1 deep link demonstrado	25%
Tema claro/escuro, layout em ≥ 2 larguras, sem overflow	20%
Acessibilidade: descrições, contraste, alvos ≥ 48 dp, leitor de tela	15%
≥ 5 testes de interface, verdes no CI	15%

Guia e tarefas: `docs/SPRINT-1.md` e `docs/SPRINT-1-TAREFAS.md`. Exemplo: `exemplos/tarefas-compose/`, passos 1 a 10.

O que muda no semestre

	Antes	Agora
Entrega da Sprint 1	02/10	**16/10** (sexta), 23:59
Depois da Sprint 1	Sprint 2, Sprint 3 e bloco final	**só a Sprint 2**, que é a entrega final, em **30/11**
Prova escrita	21/10	**09/11** (segunda), em laboratório
Prova de reposição	30/11	**02/12** (quarta)
Fim de cada sprint	apresentação por coorte	***daily meeting*** online com cada grupo, como em 28 e 30/09
Unidades	três sprints e duas provas espalhadas	U1 = Sprint 0 (30%) + Sprint 1 (70%) · U2 = prova · U3 = Sprint 2

- Dentro de cada sprint, **nada muda**: entrega técnica 50%, atividade no repositório 30%, comunicação 20%
- A *daily meeting* entra onde antes entrava a apresentação; as de 28 e 30/09 valeram para a Sprint 1
- Vale a **maior nota** entre a prova e a reposição

> Tudo está em `docs/CRONOGRAMA.md`, `docs/AVALIACAO.md` e `docs/RUBRICAS.md`.

O calendário até dezembro

Semana	Segunda	Quarta
05 e 07/10	🟢 em sala: conteúdo da Sprint 1	🟢 em sala: conteúdo da Sprint 1
12 e 14/10	feriado	🔵 online: acompanhamento · 🚀 **sexta, 16/10: entrega da Sprint 1**
19 e 21/10	🟢 em sala: conteúdo da Sprint 2	🟢 em sala: conteúdo da Sprint 2
26 e 28/10	🔵 online: acompanhamento	feriado
02 e 04/11	feriado	🔵 online: revisão para a prova
09 e 11/11	🟢 em sala: **prova escrita**	🔵 online: acompanhamento
16 e 18/11	🔵 online: acompanhamento	🟢 em sala: oficina de projeto
23 e 25/11	🔵 online: *daily meetings*	🔵 online: *daily meetings*
30/11 e 02/12	a definir · 🚀 **entrega final**	🟢 em sala: **prova de reposição**

A Sprint 2, a última

Critério	Peso
Estado e arquitetura: ViewModel com `StateFlow`, camadas, domínio sem Compose	30%
Estados da interface: carregando, erro, vazio e sucesso	15%
Dados reais e erro de rede, com Ktor Client	25%
Persistência local: os dados sobrevivem ao fechamento do app	10%
Testes de lógica: ≥ 5 testes de ViewModel no CI	10%
App e documentação: MVP sem crash, `docs/arquitetura.md`, APK numa release	10%

> O que vocês entregarem em 30/11 é o produto final. A prova de 09/11 cobre as Sprints 0, 1 e 2.

Onde estudar depois

Fonte	Foco
`leituras/moveis-s1-pte2.md`	Esta aula, com erros comuns e exercícios
`exemplos/tarefas-compose/PASSOS.md`	Passos 1 a 10
`developer.android.com/develop/ui/compose/accessibility`	Acessibilidade em Compose
`kotlinlang.org/docs/multiplatform/compose-navigation.html`	Navegação multiplataforma
`kotlinlang.org/docs/multiplatform/compose-test.html`	Testes de interface